

INSTITUTO FEDERAL
Sudeste de Minas Gerais



Comunicação entre processos (via *middleware* – especial computação alto desempenho)

Bianca Portes

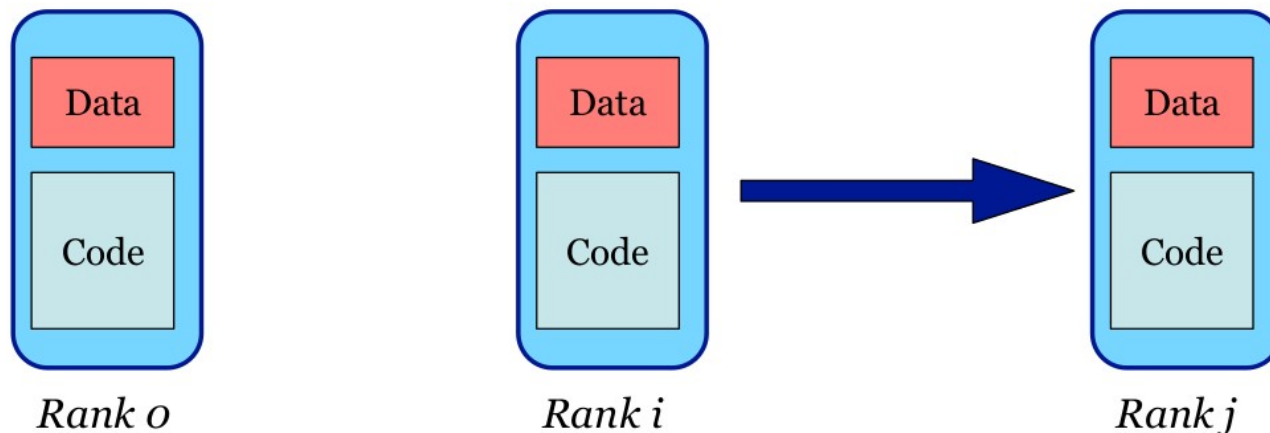
bianca.castro@ifsudestemg.edu.br

Recordando...

- Finalizamos a discussão de troca de mensagens com o padrão MPI, voltado para programação de alto desempenho
 - É um padrão flexível com centenas de operações
 - Possui diversas opções de implementação disponíveis

Recordando: nomenclaturas iniciais

- **Rank:** identificação única para cada processo
 - Inicia no 0 e é utilizado para endereçar as mensagens
- **Grupo:** conjunto de processos
 - Inicialmente, todos os processos pertencem a apenas um grupo associado a um comunicador
- **Comunicador:** identificador que define um grupo



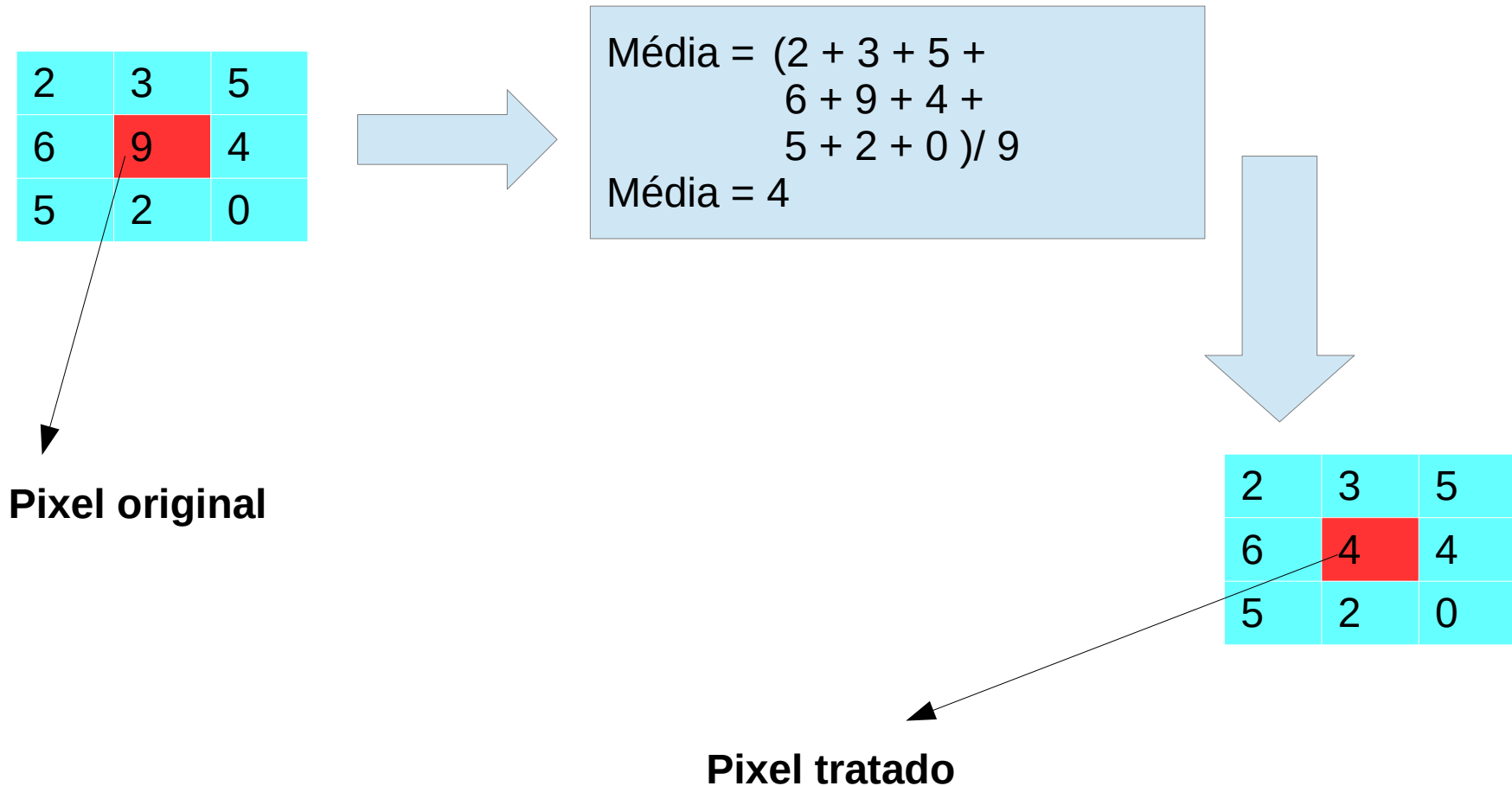
Para que servem filtros de suavização de imagens?

- Suavizar uma imagem é reduzir o nível de troca de contraste entre pixels próximos entre si
- Casos:
 - Imagens com serrilhados
 - Imagens com transições abruptas nas bordas de fundo/objeto
 - Imagens com ruídos

Exemplo de imagem com ruído



Exemplo: filtro de média 3x3

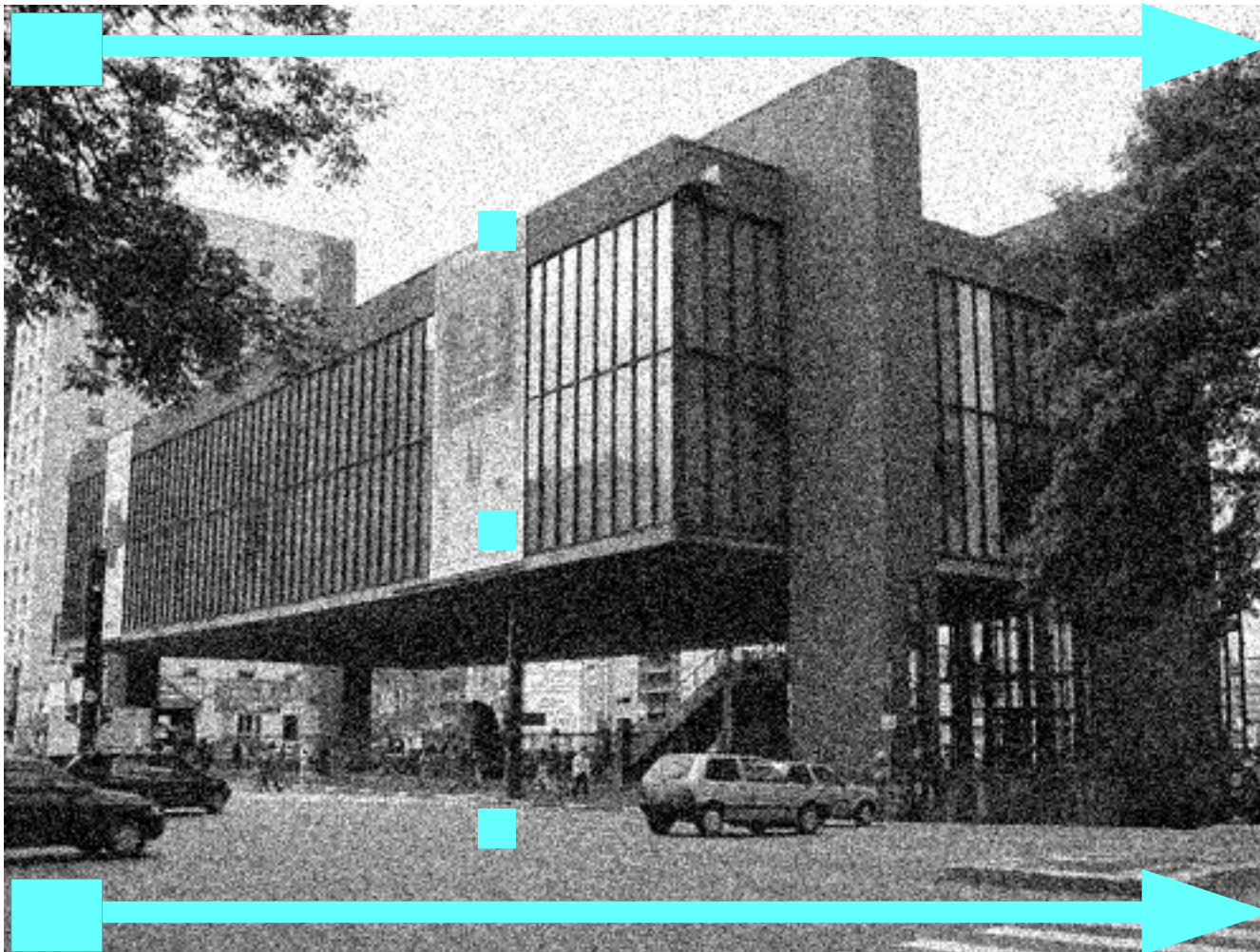


Atenção: note que as bordas da imagem não podem fazer parte do processamento!

Exemplo: filtro de média

Essa operação pontual de média tem que ser aplicada pixel a pixel na imagem: **filtragem por convolução**

Toma bastante tempo de processamento!!!!



Exemplo: filtro de média

Imagem original



Imagem tratada



- Resultado:
 - Imagens com baixo brilho e baixo contraste

OpenCV

- Para trabalharmos com imagem, usaremos o OpenCV
 - Biblioteca multiplataforma para
 - Processamento de Imagens
 - Computação Visual
 - *Open source*
 - Originalmente, desenvolvida em C/C++
 - Suportada em Python

pip install opencv-python

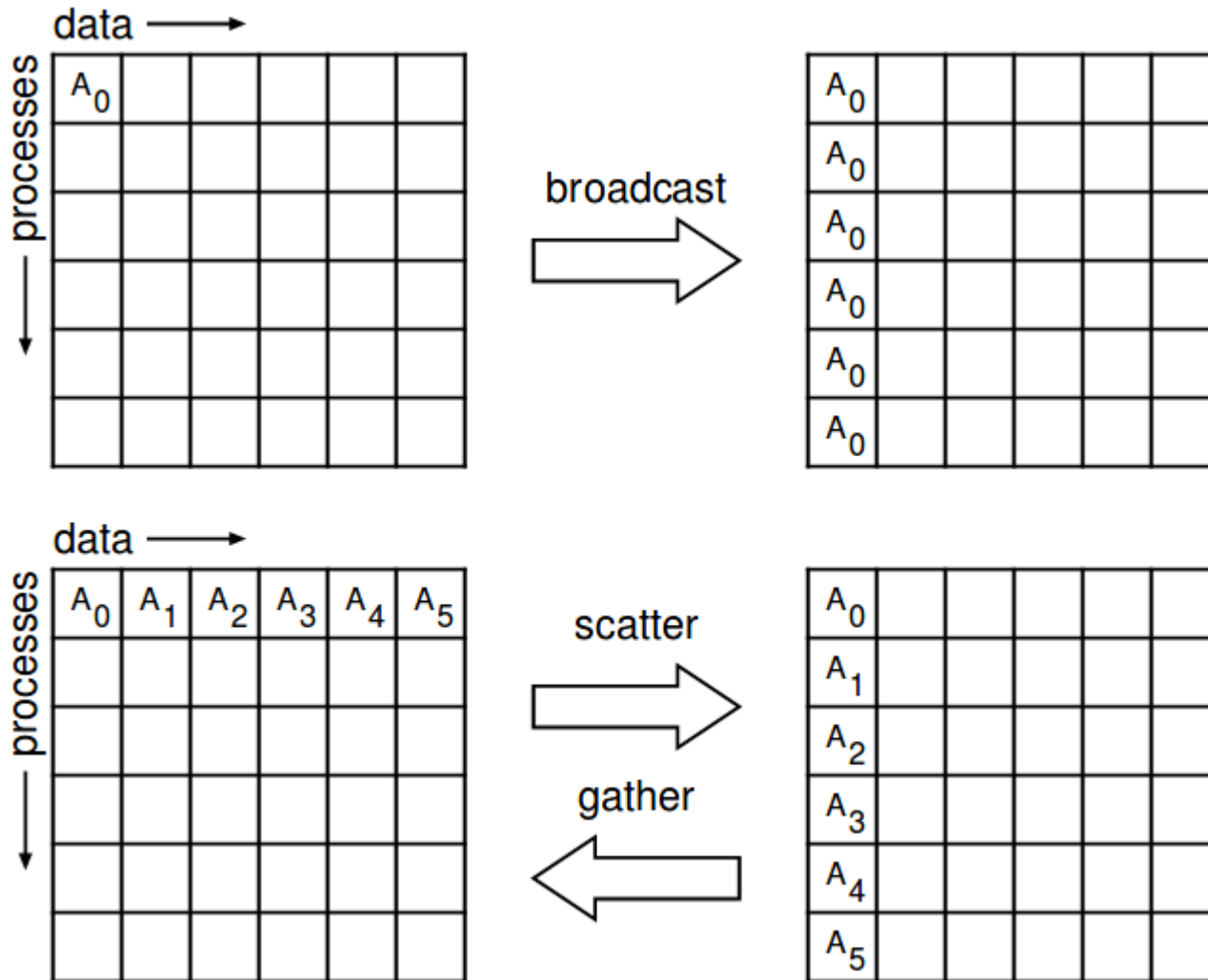


OpenCV

Implementação: filtro de média 3x3

```
1  import cv2
2  import numpy as np
3
4  image = cv2.imread("maspcomruido.jpg", 0) #trabalhando com imagem de um canal (escala de cinza)
5
6  altura, largura = image.shape
7  newimage = np.zeros((altura, largura), dtype = 'uint8')
8
9  # Filtro média 3x3
10 for y in range(1, altura - 1):
11     for x in range(1, largura - 1):
12         media = int(image[y - 1][x - 1]) + int(image[y - 1][x]) + int(image[y - 1][x + 1]) #1º linha
13         media += int(image[y][x - 1]) + int(image[y][x]) + int(image[y][x + 1]) #2º linha
14         media += int(image[y + 1][x - 1]) + int(image[y + 1][x]) + int(image[y + 1][x + 1]) #3º linha
15
16         media = int(media / 9)
17
18         newimage[y][x] = media
19
20 cv2.imshow("original image", image)
21 cv2.imshow("filtered image", newimage)
22
23 cv2.waitKey(0)
24 cv2.destroyAllWindows()
```

Operações coletivas no MPI



Otimizando o código com MPI

```
import cv2
import numpy
from mpi4py import MPI

comm = MPI.COMM_WORLD
nprocs = comm.Get_size()
myrank = comm.Get_rank()

localimage = None
newimage = None
image = None
n = 0
largura = 0

if myrank == 0:
    image = cv2.imread("maspcomruido.jpg", 0)
    altura, largura = image.shape
    n = int(altura/nprocs)
    newimage = numpy.zeros((altura, largura), dtype = 'uint8')

(n, largura) = comm.bcast((n, largura), root = 0)
localimage = numpy.zeros((n, largura), dtype = 'uint8')

#Distribuindo entre os processos
comm.Scatterv(image, localimage, root = 0)
cv2.imshow("corte", localimage) #imagens partidas

#####
#Processamento distribuído tem que ser feito aqui no meio...
#####

#Recolhendo entre os processos
comm.Gatherv(localimage, newimage, root = 0)

if myrank == 0:
    cv2.imshow("original image", image)
    cv2.imshow("filtered image", newimage)
    cv2.waitKey(0)

cv2.destroyAllWindows()
```

Faça essa parte! Implemente a lógica da divisão do trabalho!

Otimizando o código com MPI

```
import cv2
import numpy
from mpi4py import MPI

comm = MPI.COMM_WORLD
nprocs = comm.Get_size()
myrank = comm.Get_rank()

localimage = None
newimage = None
image = None
n = 0
largura = 0

if myrank == 0:
    image = cv2.imread("maspcomruido.jpg", 0)
    altura, largura = image.shape
    n = int(altura/nprocs)
    newimage = numpy.zeros((altura, largura), dtype = 'uint8')

(n, largura) = comm.Bcast( (n, largura), root = 0)
localimage = numpy.zeros((n, largura), dtype = 'uint8')

#Distribuindo entre os processos
comm.Scatterv(image, localimage, root = 0)
cv2.imshow("corte", localimage) #imagens partidas

#####
#Processamento distribuído tem que ser feito aqui no meio...
#####

#Recolhendo entre os processos
comm.Gatherv(localimage, newimage, root = 0)

if myrank == 0:
    cv2.imshow("original image", image)
    cv2.imshow("filtered image", newimage)
    cv2.waitKey(0)

cv2.destroyAllWindows()
,
```

Este código tem uma limitação nas bordas. Você consegue identificar qual?

Vamos praticar!!!

1) Complete o algoritmo anterior de forma a aplicar a filtragem de média por convolução.

- Rode com apenas 5 processos
- Calcule o *speedup*

Regra de ouro para desempenho no mpi4py

A escolha entre maiúscula e minúscula não é estilo. É desempenho!

⊗ minúscula — objeto Python

```
comm.send(minha_lista)
```

- Python serializa o objeto via pickle
- dado vira bytes, trafega, e é reconstruído no destino
- muitas camadas extras → muito lento

✓ Maiúscula — buffer numpy

```
comm.Send(meu_array_numpy)
```

- mpi4py entrega o endereço de memória do bloco C direto à placa de rede
- sem serialização — transmissão equivalente ao C
- requer array numpy — não funciona com listas Python

Resumindo:

OBJETO PYTHON

Python → pickle → bytes →
rede → unpickle → Python



BUFFER NUMPY

memória C → rede →
memória C

Sempre use maiúsculas para transmissão de dados volumosos, como imagens, vetores e matrizes

Vamos praticar!!!

2) Valide a regra de ouro comparando *bcast* vs *Bcast*.

Implemente um script de *benchmark* para observar o impacto da serialização Python vs comunicação direta via *buffer*. Para isto, considere:

- **warm-up**: neste contexto, seria rodar a função de comunicação uma primeira vez antes de começar a disparar o cronômetro
- **sincronização**: implemente *comm.Barrier()* antes e depois de cada chamada para medir o tempo real da rede. Ou seja, até que TODOS os nós cheguem na mesma linha de código
- **isolamento de ruído**: para evitar que ruídos relacionados ao SO, como a inicialização do ambiente MPI via *mpiexec*, utilize o loop de 30 iterações dentro do script Python
- remova os *outliers*

Vamos praticar!!!

3)Faça uso das operações coletivas do MPI para calcular a média de intensidade dos pixels (ou média dos valores de todos os pixels).

Se mais próximo de 255, a imagem é clara/brilhante

Se mais próximo de 128, o tom é de cinza médio

Se mais próximo de 0, a imagem é escura

- MPI4PY referência:

<https://mpi4py.readthedocs.io/en/stable/tutorial.html>